Investment Codex

Practical prompt catalogue

34 copy-ready prompts • Required inputs • Native outputs • Execution limits

Implemented Python 0.6.0 checkout; published release e88446f; documentation/preflight through Phase 6. Not a current market assessment.

Audience: Ahmed • Timezone: Asia/Karachi • Prepared: 1 November 2026

Use this guide to request implemented capabilities without confusing a conversational instruction, a submitted request and an accepted financial result.

Actual engine output: JSON, Markdown receipts and Excel workbooks. The code has no native PDF exporter. PDF delivery is an assistant/reporting step using verified outputs; appending “PDF” to a request does not make the runner generate one.

Operational state: Phase 5 code is published and passed 392 CI tests. Phase 6 production activation remains blocked; the accepted workspace is DEMO, not Ahmed’s real portfolio. This guide makes no live market assessment.

Cloudflare: investmentcodex is the chosen bucket. Authenticated small text upload/download and SHA-256 verification passed. Public r2.dev access remains enabled by user choice. Private financial files must not be uploaded there.

Automation: the Python runner still uses separately configured boto3 credentials. ChatGPT Cloudflare plugin authentication is not inherited by GitHub Actions; binary-workbook plugin transport and a complete plugin-backed runner are unverified.

1. How to use this catalogue

Replace angle-bracket placeholders with your actual instruction. Give only the changed facts if the accepted workspace already contains valid inputs; the assistant must reuse those records first. When supplying an update, the serialized owned collection must still be complete and schema-valid.

Use the latest accepted Investment-Codex state first. Resolve dataset_id and published_run_id from committed data/canonical/portfolio.json. Use <date> explicitly in Asia/Karachi and preserve the original market cutoff. Tell me exactly which fields/sources are missing rather than fabricating them. Use simulation for what-if requests; accept new inputs only when I explicitly ask. Produce a PDF report and link JSON, Markdown and Excel evidence. If execution is disabled, prepare/validate the request and say submitted or blocked, never completed.

Read-only versus derived runs

Readiness, validation and comparison commands inspect committed evidence. Most run commands recalculate and publish derived state/workbooks even if no new financial input is accepted. Simulation instead calculates in an isolated checkout and publishes scoped results; validate must not be mistaken for acceptance.

Submission routes

The workspace envelope supports nine operations and eleven workflows. It does not accept shell commands, credentials, arbitrary file paths or a ticker filter. Some operator capabilities—migration, dataset activation, CSV import, workbook notes and benchmarking—are separate CLI entry points. CLI examples in this guide are operator routes, not fields to paste into a request.

Gates and statuses

Private Actions request processing and production/storage policies remain disabled until prerequisites pass. Reversible inspection and synthetic CLI demonstrations do not establish production readiness. Return module and transfer statuses independently; COMPLETE means the selected calculation is calculable, not approved/profitable or deployed.

Imports and ownership

A new source or record ID is required for revised evidence, with explicit supersession where applicable. Never edit accepted transaction IDs or omit accepted owned records to delete them. Partial data or unknown required assumptions produce diagnostic gaps, not fabricated answers.

2. Prompt finder

ID	What to ask for
P01	Check readiness and missing inputs
P02	Run an isolated complete synthetic demonstration
P03	Reconcile and review my existing portfolio
P04	Accept a revised portfolio or price snapshot
P05	Analyze one already registered security
P06	Run the required valuation screen
P07	Import reported and normalized earnings with forecasts
P08	Check a Bank, E&P or Industrial numerical model
P09	Revise shared scenario assumptions
P10	Simulate a forecast or portfolio change
P11	Reconcile dividends and entitlement corrections
P12	Run RS, trend, sector rotation and mean reversion
P13	Review macro observations and scenarios
P14	Import a news or earnings event packet
P15	Review portfolio concentration and sizing constraints
P16	Update or review the watchlist
P17	Prepare IC proposals
P18	Record an explicit IC approval, rejection or deferral
P19	Project retirement and liquidity
P20	Compare retirement what-if choices
P21	Compare PSX, gold, USD, funds and optional crypto
P22	Explain what changed between committed runs
P23	Inspect and reconcile the legacy workbook
P24	Import approved evidence and original documents
P25	Fetch and verify an R2 source artifact
P26	Accept a previously verified transferred package
P27	Archive a committed analytical run to R2
P28	Inspect a failed transfer and retry only storage

ID	What to ask for
P29	Validate, export or replay a workbook
P30	Activate an explicitly approved real dataset
P31	Review or import permitted workbook notes
P32	Run a budgeted refresh or measure caching
P33	Recover publication, submit pending requests and publish
P34	Check automation prerequisites without starting schedules

3. Inputs: what to provide and in which format

A - Portfolio

Full Bundle JSON for initial import: mode, market_cutoff, sources, securities, opening, closing, transactions, prices; dataset_id and policy as applicable. Snapshot = timezone-bearing timestamp, source_id, cash, positions[{ticker,quantity,cost}], where cost is total position cost. Transactions need immutable ID, timestamp, settlement_date, kind and source_id; BUY/SELL need ticker/quantity/price, cash flows amount, SPLIT ratio. Prices need ticker/session_date/close/source_id. Workspace updates use package.portfolio (complete securities/opening/closing/transactions/prices; optional policy) plus package.sources. See docs/input-schema.json and WorkspacePackage schema.

B - Research and forecasts

JSON research contains universe/expiry verification, earnings, models, dividends and resolutions. Supply sourced reported PAT/shares/EPS, one-off signed adjustments and tax effects; owned model/method/version, accounting/period/share bases, benchmark/ERP, bear/base/bull drivers and applicable approval. Current real core-sector certification needs numerical sector_forecasts, financials and shared scenario mappings. Source references need locators and confirmed evidence. Unknown required drivers block calculation. See research-schema.json and financial-fact-schema.json.

C - Integration and history

JSON integration holds calendar, memberships, histories, timing_policy, macro, macro_scenarios, events, risk_limits, factors, catalysts, candidates, proposals and alert_methodology. Records have record_id/ticker/effective_date/valid_until/source_ids; corrections supersede old IDs. Supplied collections are complete owned views, not deletion patches. OHLCV CSV is supported only via import-history with a companion JSON packet. All timing/risk parameters, source provenance and corporate actions must be explicit. See integration-schema.json.

S / D - Scenarios and decisions

ScenarioProfile supplies version/name/basis/period/horizon, dated sources and numerical assumptions with units. ScenarioLink supplies profile_id/basis/period/horizon, numerical bindings and binding_units; sourced rationale/overrides for domain differences. ICDecision supplies proposal_id, decision, actor/time, reason and reviewed_basis_hash; approved_quantity/transaction_ids only for APPROVE. See scenario-profile-schema.json and ic-decision-schema.json.

W - Wealth

A sources/wealth JSON packet or package.wealth. Assets need unique asset/economic IDs, kind, balance or accepted portfolio_link, currency, availability and explicit USD FX where needed. Flows specify net amounts, nominal/real basis, destination, frequency, dates and unique benefit claim IDs. Scenarios specify inflation and per-asset return/fee/tax rates; plans specify month-start dates, months, spending and complete withdrawal order. Histories specify ordered quotes, FX and price/total-return basis. See wealth-schema.json and wealth-packet-schema.json.

E / M / T - Evidence, migration, storage

EvidencePackage 4.0 imports sources, structured owned modules and licensed original descriptors; CLI file paths resolve relative to its package. Workspace originals use verified receipt_id/artifact_id plus EvidenceDocument, never arbitrary paths. Migration requires XLSX plus WorkbookMigration JSON with hash/mappings/approval/permission/tolerances. StorageRequest JSON accepts only pull or push, not shell commands. See evidence-package-schema.json, migration-schema.json and storage-request-schema.json.

Common record conventions

Research/decision/fact records generally require record_id, ticker (or declared aggregate key), effective_date, valid_until and source_ids. Use YYYY-MM-DD for date-only fields and timezone-bearing ISO timestamps, e.g. 2026-11-01T10:00:00+05:00. Decimal strings preserve explicit financial precision. Fractions are proportions: 0.20 means 20%, not 20. All examples/templates are placeholders or synthetic references, never Ahmed's holdings.

Raw OHLCV CSV header

```
session_date,open,high,low,close,volume,traded_value
```

Header names AND order must match. JSON supplies history ID/ticker, sources, completed calendar, membership, action coverage, corporate actions and timing policy. A generic broker CSV or arbitrary spreadsheet is not directly accepted as a portfolio package.

4. Numerical engine input checklist

Bank

Average earning/funding balances and annual rates; fees, treasury income, operating costs, credit charge and tax charge; shares; opening equity, OCI, capital movements; opening eligible capital, eligible OCI, RWA, minimum capital ratio, distributable reserves, available cash and proposed dividends. Reconcile EPS, ROE, book value and DPS. OCI remains outside PAT.

E&P

For each commodity: physical unit, opening reserves/additions, production/inventory release/sold volumes, realized price/currency, explicit FX and operating cost per physical unit. Plus royalties/fixed costs/depletion/interest/other income/tax, opening receivables/collections, other working capital explicitly excluding receivables, capex, borrowing and cash-income flag; shares and enforced payout cross-check.

Industrial

For each product: unit, volume, PKR net price and variable cost per unit. Plus fixed costs, depreciation, interest, other income, explicit tax, receivables/inventory/payables movements, capex, borrowing, shares, cash-income flag and supplied mid-cycle rationale.

Common checks

All required typed drivers must be non-null. Bound facts must match values, accounting basis, dates and units. Per-unit facts need quantity_unit matching volumes; USD conversion uses PKR_PER_USD and PKR identity conversion uses FRACTION=1. Typed forecasts use contiguous FY/CY annual grids with consistent year width; NTM supports one explicitly labelled annual forecast. DCF projection length is separate from investment holding horizon. No guessed commodity conversion or implicit currency/scale normalization.

Retirement-specific pitfalls

Current balances must not also be counted as future benefit flows. A PSX_SLEEVE links to the accepted portfolio instead of copying balances; TOTAL cannot overlap EQUITY/CASH links. Benefit economic_claim_id values must be unique. Spending and contributions need net amounts and nominal/real bases. Every plan's withdrawal order covers each asset exactly once. Dates start on the first of the month; up to 1200 projection months are supported. Crypto is explicit opt-in, not a default.

5. Copy-ready prompts

P01 Check readiness and missing inputs

Read the accepted workspace and report what is ready, blocked, stale or missing for portfolio, valuation, timing, macro, IC, retirement and cross-asset workflows. Distinguish software readiness from production readiness. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: No new financial data. Reuse committed canonical state and configuration; specify the intended assessment date if a new dated calculation is wanted.

Execution route: CLI: readiness; schedule-status. These read committed status, not a fresh market assessment.

Native output formats: Readiness JSON: per-workflow coverage, compatibility, deployment status, dependency reasons and next actions. Schedule-status JSON.

State effect / limitations: Read only. A new date is not silently applied to this read-only command.

Code: readiness.py:committed_report; cli.py

P02 Run an isolated complete synthetic demonstration

Create a separate empty demo workspace using the completion fixture and demonstrate the implemented scenarios, numerical forecasts, portfolio and wealth modules. Do not replace my accepted workspace. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: An empty separate directory; supplied completion-demo fixture. No personal balances or secrets.

Execution route: CLI: --root <empty_demo> init --offline --completion-demo.

Native output formats: Synthetic JSON state, run manifest, coverage/validation/change/usage files and Investment_Workspace.xlsx.

State effect / limitations: Creates a DEMO only in the selected empty workspace. Never initializes over accepted data.

Code: cli.py:init; pipeline.py:fixture_bytes

P03 Reconcile and review my existing portfolio

As of <date>, reconcile accepted holdings, cost and cash; explain realized/unrealized returns, stale prices and exceptions. Keep dividends receivable separate from received cash. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack A already accepted: opening/closing dated snapshots, ledger, securities, sources and prices; optional declared tolerances/risk policy.

Execution route: Workspace run with workflows [portfolio], or CLI run portfolio --offline --as-of <date>.

Native output formats: Holdings, Transactions, Cash, Reconciliation and Data_Gaps sheets; canonical snapshot and reports/latest/summary.json.

State effect / limitations: Recalculates/publishes derived reports; does not import a new balance package. DEMO portfolio readiness can remain PARTIAL.

Code: portfolio.py:calculate; readiness.py:workflow_status

P04 Accept a revised portfolio or price snapshot

Validate this approved complete portfolio snapshot first. Then accept the supplied update with new source identities where evidence changed, reconcile it and recalculate portfolio and valuation. Do not alter other module-owned inputs. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack A as WorkspacePackage.portfolio plus sources. Supply the complete active portfolio view, not one-cell patches. A revised price feed uses appropriate new source IDs; do not modify accepted transaction IDs.

Execution route: Workspace validate first; explicit accept-and-run/import with package.portfolio. CLI import-portfolio accepts a full Bundle, not the same partial workspace envelope.

Native output formats: Validation/request receipts; accepted canonical state, source history, run manifest, refreshed workbook and scoped request snapshot/workbook.

State effect / limitations: Explicit acceptance changes financial inputs only after validation. Initial real dataset activation is P30.

Code: workspace_requests.py:prepare/execute; pipeline.py:run

P05 Analyze one already registered security

Analyze <ticker> using its accepted earnings, sector model, scenarios, price, timing, events and risk basis. Explain valuation, entry gates, catalysts, invalidation and missing evidence. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Ticker must exist in securities. Packs B and C, current price and applicable approvals must already be accepted, or be supplied separately for validation/import.

Execution route: CLI analyze <ticker> --offline. Alternatively run valuation/timing/ic and present that ticker; the workspace envelope has no ticker-filter field.

Native output formats: Ticker-filtered CLI security JSON plus shared-workspace outputs; Earnings_Bridge, Valuation, Entry_Levels and integration sheets where populated.

State effect / limitations: Recalculates shared dependencies/full required coverage; not an isolated ticker-only calculation or automatic company data download.

Code: cli.py:analyze/run; research.py

P06 Run the required valuation screen

As of <date>, screen the dated required universe and give every ticker a final coverage outcome. Rank only eligible validated base-case valuations, show fewer than ten if necessary, and explain all exclusions. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack B: verified universe/membership, earnings, models, bear/base/bull assumptions, current prices, sovereign benchmark/ERP, financial facts, scenario mappings and approvals. Pack C supports

timing/risk gates.

Execution route: Workspace run [valuation]; CLI run valuation --as-of <date> --offline --require-full-coverage.

Native output formats: Coverage register, valuation/return/entry tables, eligible Top_10, data gaps, valuation summary JSON and workbook.

State effect / limitations: Publishes derived results. Incomplete models produce PARTIAL/BLOCKED outcomes; a full coverage ledger alone is not a complete screen.

Code: research.py:_calculate; models.py:reference_value

P07 Import reported and normalized earnings with forecasts

Validate the supplied filing-derived earnings bridge and revised owned model for <ticker>. Show reported, normalized and forward PAT/EPS, signed adjustments, taxes, share basis and scenarios before accepting the approved package. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack B: Earnings and ValuationModel records, source locators, adjustment rationale/tax effect, accounting/period/share bases, typed drivers and version-matched approvals. Supersede existing records explicitly.

Execution route: Workspace validate/simulate package.research + financials + sources, followed by explicit import/accept-and-run. CLI import-research only accepts sources/research; use import-evidence for a richer combined pack.

Native output formats: Earnings_Bridge, Forward_Bridge, Valuation and coverage diagnostics; validated/accepted input and request receipts.

State effect / limitations: No automatic PDF-to-financial-statement parser; the backend consumes controlled structured inputs.

Code: schemas.py:Earnings/ValuationModel; intake.py:EvidencePackage

P08 Check a Bank, E&P or Industrial numerical model

Validate the supplied <Bank/E&P/Industrial> forecast against its sourced facts. Show intermediate calculations, PAT/EPS, FCFE where applicable, capital/payout constraints, units, share basis and period reconciliation. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack B engine records; see the engine checklist and required-field appendix. Every required driver must be supplied; bound facts need canonical units, physical denominators and explicit FX as applicable.

Execution route: Workspace validate or simulate [valuation] with revised research/financials; certification uses the normal valuation engine.

Native output formats: Forward_Bridge numerical intermediates, valuation outputs and specific blocking reasons. Bank has equity/dividend-capacity outputs; FCFE is for E&P/Industrial.

State effect / limitations: Validation/simulation does not accept inputs. Other sectors do not have these first-wave typed engines.

Code: sector_engines.py:forecast; forecast_contracts.py; models.py

P09 Revise shared scenario assumptions

Validate these new versioned bear/base/bull profiles and explicit valuation/macro/wealth mappings. Show which dependent forecasts need revision, and preserve owned EPS until an approved model update is imported. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack S: profiles with basis, period, horizon, assumptions/units, dates and evidence; mappings with bindings and units; sourced rationale for overrides/domain differences; revised owned records for intentional acceptance.

Execution route: Workspace validate/simulate package.scenario_profiles plus affected research/integration/wealth. Accept only with explicit import.

Native output formats: Scenario_Profiles, mapped scenario diagnostics, forecast/wealth results and data gaps for stale/mismatched dependencies.

State effect / limitations: Profile changes do not automatically rewrite EPS or carry short-term assumptions into retirement horizons.

Code: scenarios.py:prepare/resolve; schemas.py:ScenarioLink

P10 Simulate a forecast or portfolio change

Simulate <specific supplied changes> as of <date> against accepted inputs. Compare valuation, portfolio risk and/or retirement outcomes, and leave canonical financial inputs unchanged. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Only changed owned modules in a complete typed WorkspacePackage; use new IDs/supersession for revised accepted records, explicit assumptions, and expected dataset/run IDs.

Execution route: Workspace simulate with selected workflows and optional package or verified package_artifact.

Native output formats: reports/requests/<id>/Investment_Workspace.xlsx, snapshot.json, JSON/Markdown receipt, module statuses and missing-input list.

State effect / limitations: Isolated calculation; accepted_inputs_changed must be false. There is no arbitrary code/formula execution or generic numerical optimizer.

Code: workspace_requests.py:execute

P11 Reconcile dividends and entitlement corrections

Update dividends as of <date>. Show historical entitlement, linked receipts, gross/net pending, overdue balances and withholding exceptions. For a correction, preserve old proof and recalculate only from verified revised evidence. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack B dividends plus Pack A opening snapshot, settled quantity-affecting transactions and accepted DIVIDEND receipt IDs; ex/record/payment dates, DPS, tax policy and sources. Use revision_kind ENTITLEMENT_BASIS or RECEIPT_UPDATE for revisions.

Execution route: Workspace run [update-dividends] for accepted data; validate/import a revised research/portfolio package when new facts are supplied. CLI update-dividends has no --as-of; use run update-dividends --as-of <date>.

Native output formats: Dividend_Calendar: eligible quantity, immutable entitlement basis, gross/net entitlement/received/pending, overdue/status flags and correction proof; gaps/receipts.

State effect / limitations: Receivables never increase brokerage cash. Unknown tax gives null net pending; fees do not create unpaid dividend entitlement.

Code: dividends.py:result; research.py:dividend_result

P12 Run RS, trend, sector rotation and mean reversion

Using verified completed-session data, identify leaders, laggards, sector rotation, trend stages, extension and mean-reversion conditions. Disclose calibration and backtest limitations. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack C: calendar, dated membership, benchmark/security OHLCV, corporate actions/action coverage, sector labels and explicit TimingPolicy windows, thresholds and liquidity/calibration parameters.

Execution route: Workspace run [timing]; CLI run timing. To import CSV: import-history --file <csv> --packet <integration.json> --history-id <id>.

Native output formats: RS_Trend, Sector_Rotation, Mean_Reversion, Backtests/Backtest_Trades and gaps; feeds/latest/timing.json.

State effect / limitations: No inferred histories or live-feed discovery. CSV header order is exact; JSON provides provenance, calendar and actions.

Code: timing.py; workflows.py:_calculate; adapters.py:read_history_csv

P13 Review macro observations and scenarios

Review the accepted Pakistan/global macro observations and bear/base/bull paths as of <date>. Separate facts, interpretations and assumptions, identify conflicts/expiry and flag affected forecasts for owned revision. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack C macro observations with indicator, value/unit, narrative, dates and sources; macro scenarios and Pack S mappings. Numerical claims require supplied evidence.

Execution route: Workspace run [macro]; import revised sources/integration separately if requested.

Native output formats: Macro, Scenarios, News_Impact/Event_Propagation where relevant, refresh queue and macro completeness status.

State effect / limitations: No autonomous macro feeds or guaranteed economic forecasts. Numerical alert evaluation requires approved methodology and supplied scores.

Code: workflows.py; schemas.py:MacroObservation/MacroScenario/AlertMethodology

P14 Import a news or earnings event packet

Validate and import this approved event packet. Map the stated transmission to the supplied affected tickers, report urgency, and queue stale owned models rather than silently applying an EPS change. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack C complete integration update plus new sources; event headline, affected_tickers, transmission, urgency, requires_model_revision. If eps_delta is given, include eps_period and eps_calculation.

Execution route: Workspace validate then import/accept-and-run [event]; CLI run event --source-file <sources-integration.json> --as-of <date>.

Native output formats: News_Impact, Event_Propagation, Refresh_Queue, changed coverage/readiness, run/request receipts and workbook.

State effect / limitations: Acceptance changes the integration inputs. A headline/PDF alone is not a valid event packet; no automatic owned EPS overwrite.

Code: schemas.py:NewsEvent; workflows.py:event propagation; cli.py:run

P15 Review portfolio concentration and sizing constraints

Review position, sector and economic-factor concentration, cash, correlations, scenario returns and research quantity caps. Explain breached limits and missing inputs. Do not generate orders. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Packs A/B/C: reconciled current holdings/prices, supplied user-confirmed RiskLimits, factors, histories/calendar, valid valuations and candidate entry/stop/target assumptions.

Execution route: Workspace run [ic] and/or [daily]; CLI run ic. Both operate on shared accepted inputs.

Native output formats: Portfolio_Risk, Correlations, IC_Actions sizing details, scenario coverage, proposed/illustrative caps and gap reasons.

State effect / limitations: Calculated caps are not trades. DEMO data and incomplete risk/history coverage can keep gates blocked.

Code: workflows.py:risk_outputs; decisions.py:basis

P16 Update or review the watchlist

Review the watchlist for approaching entries, stale theses, catalysts and invalidation. For the supplied update, validate the revised complete candidate collection before accepting it. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack C WatchCandidate: author, thesis, invalidation, portfolio_fit, desired_entry, stop_price, approach_fraction, next_review, requested stage/weight and catalyst IDs; Pack A/B/C for derived readiness.

Execution route: Workspace run [watchlist] for monitoring; validate/import package.integration for changes.

Native output formats: Watchlist, Catalysts, gate/monitoring states, next-review information and IC/risk dependencies.

State effect / limitations: No standalone add/remove command. Updates use owned records; omission is not deletion—use supersession and requested archive/reject stages.

Code: schemas.py:WatchCandidate; workflows.py:watchlist

P17 Prepare IC proposals

Review supplied IC proposals against valuation, timing, candidate readiness, portfolio risk, invalidation and opportunity cost. Clearly distinguish proposals from user decisions. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack C ICProposal records with action, reason, confidence, invalidation, opportunity_cost and candidate_id where applicable; accepted model/risk/timing/candidate evidence.

Execution route: Workspace run [ic]; explicit package.integration import only if proposal records are changing.

Native output formats: IC_Actions with proposed/effective research action, readiness reasons, sizing cap and supporting tables.

State effect / limitations: Derived proposal actions never constitute user approval, execution or an order.

Code: schemas.py:ICProposal; workflows.py:IC actions

P18 Record an explicit IC approval, rejection or deferral

Record my explicit <APPROVE/REJECT/DEFER> decision for <proposal_id> with my reason, date and reviewed basis. If I supplied a completion quantity and imported transaction IDs, report factual fulfilment separately. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack D: ICDecision actor/timezone timestamp, effective/expiry dates, reason, source IDs, proposal ID and exact reviewed_basis_hash from decision-basis. APPROVE may include approved_quantity and valid transaction IDs.

Execution route: CLI decision-basis to obtain current hashes; workspace operation decision with package.sources and package.decisions. Do not substitute an ICProposal.

Native output formats: IC_Decisions, immutable history, CURRENT/NEEDS_REVIEW and execution-fulfilment statuses; request receipt.

State effect / limitations: Changes accepted user decision records, not trades. Material risk/source versions can require review even inside limits.

Code: decisions.py:basis/calculate; schemas.py:ICDecision

P19 Project retirement and liquidity

Project my supplied plan under bear/base/bull assumptions. Show nominal/real wealth, accessible versus locked assets, spending gaps, benefit timing and depletion dates. State all unknown assumptions. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack W: dated asset balances and accessibility, currencies/FX, net contribution/benefit/expense flows, inflation and per-asset return/fee/tax policies, month-start plan dates, horizon and withdrawal order; shared long-term mapping for real certification.

Execution route: Workspace run [retirement] with accepted wealth; CLI import-wealth for an approved sources/wealth packet, or workspace validate/import a wealth package.

Native output formats: Retirement monthly trajectories, Retirement_Summary, Wealth_Assets/Flows and model/data gaps; JSON snapshot and workbook.

State effect / limitations: Deterministic scenario projection, not Monte Carlo success probability. Missing PSX inputs need not block independently supplied wealth assets.

Code: wealth.py:project/calculate_wealth; schemas.py:Wealth

P20 Compare retirement what-if choices

Simulate the supplied contribution/spending/retirement-date alternatives <A and B> against the same accepted base. Compare real balances, funding gaps, accessibility and depletion without accepting either alternative. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack W for each alternative, including superseding scenario/flow/plan IDs as needed, explicit inflation/returns/tax/fees and complete updated owned collections.

Execution route: Two workspace simulate requests with unique IDs and workflows [retirement]; or compatible legacy wealth requests. wealth-preflight checks a candidate packet without changing the caller checkout.

Native output formats: Scoped snapshots/workbooks and receipts for each alternative; assistant-produced comparison PDF from those results.

State effect / limitations: No built-in contribution search/retirement optimizer. A/B comparison is orchestration of supplied deterministic scenarios.

Code: wealth_requests.py:preflight/sandbox; workspace_requests.py:simulate

P21 Compare PSX, gold, USD, funds and optional crypto

Compare the supplied asset histories on synchronized common dates and a consistent PKR basis. Show returns, observations and pairwise correlations, with explicit price versus total-return limitations. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack W assets/histories: asset_key, currency, PRICE/TOTAL_RETURN basis, ordered dates/quotes, dated explicit USD FX and source IDs; minimum observation setting. Crypto requires include_crypto=true.

Execution route: Workspace run [cross-asset], or simulate a revised supplied wealth package.

Native output formats: Cross_Asset, Asset_Correlations and coverage/insufficient-history diagnostics; JSON and workbook.

State effect / limitations: No live quote acquisition or automatic optimized allocations. Price histories must not be silently called total-return histories.

Code: wealth.py:compare_histories; schemas.py:CrossAssetHistory

P22 Explain what changed between committed runs

Compare the current accepted state with run <run_id>. Explain changes in owned inputs, results, coverage and decisions, distinguishing data changes from unchanged carried calculations. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Existing committed historical run ID; no new financial input. Resolve the comparison base explicitly.

Execution route: CLI changes --since-run <run_id>; inspect committed run manifests/snapshots and changes records.

Native output formats: Comparison JSON plus reports/latest/changes.md and run changes.json where already published; narrative PDF.

State effect / limitations: Read only. Do not invent business causes beyond recorded evidence or imply every change has a live-news explanation.

Code: pipeline.py:changes_since

P23 Inspect and reconcile the legacy workbook

Inspect <workbook.xlsx> read-only, inventory sheets/formulas and prepare the supplied mapping/parity dry run. List missing mappings, ignored-sheet reasons and model conflicts before any acceptance. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack M: actual XLSX and WorkbookMigration JSON with SHA-256, sheet/column/key mappings, approved_by, reason, store permission, tolerances and methodology-review status.

Execution route: CLI inspect-datastore --workbook <xlsx>; migrate-workbook --file <xlsx> --mapping <mapping.json> --dry-run. --accept is a separate explicit action.

Native output formats: Sheet/formula inventory JSON, reconciliation differences and migration diagnostics; accepted migration record only after explicit acceptance.

State effect / limitations: Inspection is not recovered methodology approval or certified parity. Never execute workbook formulas or manually merge binary workbooks.

Code: adapters.py:inspect_datastore; intake.py:migrate_workbook; migration.py

P24 Import approved evidence and original documents

Validate this controlled evidence package, retain only licensed-to-store originals, and then accept the explicitly approved records with complete provenance. Identify unsupported or missing extraction fields. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack E: EvidencePackage schema_version 4.0; sources and optional bundle/research/integration/wealth/financials/profiles/decisions; files with path and EvidenceDocument source/hash/suffix/role/store_permitted. Paths are for CLI operator imports only.

Execution route: CLI import-evidence --file <package.json> --as-of <date>. Workspace requests instead use package.originals verified transfer references; no arbitrary local paths.

Native output formats: Evidence_Library, content-addressed evidence blobs, source registry, accepted records/workbook and run receipt.

State effect / limitations: Raw PDF/XLSX files are stored/referenced, not automatically converted into verified financial facts.

Code: intake.py:EvidencePackage/import_package; schemas.py:EvidenceDocument

P25 Fetch and verify an R2 source artifact

From investmentcodex, fetch approved/<exact_key> with expected SHA-256 <hash>. Verify bytes, preserve source metadata and stage the artifact. Do not accept it as financial input. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack T pull: unique request_id, operation pull, objects with exact permitted keys and preferably expected sha256. Active transport and required connection policy must be available.

Execution route: Workspace storage.pull or CLI storage pull --file <storage.json> [--dry-run]. Interactive Cloudflare plugin is a separate operator route; it is not called by the Python runner.

Native output formats: reports/storage/<id>.json; artifact_id/hash/size/pinned status; ignored .runtime/r2/artifacts staging. Workspace JSON/Markdown receipt.

State effect / limitations: Only sources/ and approved/ are configured for pulls. Unpinned or failed/partial receipts cannot feed accepted imports.

Code: object_storage.py:transfer_receipt/artifact; workspace_requests.py

P26 Accept a previously verified transferred package

Validate the workspace package referenced by receipt <receipt_id> and artifact <artifact_id>. After my explicit approval, import its typed records and recalculate the selected modules. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: A COMPLETE pinned pull receipt plus matching artifact bytes containing a valid WorkspacePackage; source IDs, dataset/run revision and acceptance authorization.

Execution route: Workspace validate/simulate first; explicit import or accept-and-run with package_artifact={receipt_id,artifact_id}.

Native output formats: Validation/request receipts and, after acceptance, canonical records, source history, run artifacts and workbook.

State effect / limitations: No arbitrary downloaded file is accepted: it must match the package contract. Original PDFs/XLSX use package.originals and EvidenceDocument descriptors instead.

Code: workspace_requests.py:prepare; object_storage.py:artifact

P27 Archive a committed analytical run to R2

Archive committed run <run_id> to investmentcodex using only these exact manifest paths: <paths>. Verify read-back SHA-256 and write the archive manifest last. Report storage status separately from financial acceptance. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Pack T push: unique request_id, operation push, committed run_id, exact paths in its manifest; enabled authenticated runner storage route, prefixes and limits.

Execution route: Workspace storage.push or CLI storage push --file <storage.json> [--dry-run]. An accepted calculation may request optional archive_paths.

Native output formats: Immutable archives/<run_id>/<sha256>/<filename> objects, verified archive manifest and durable storage receipt.

State effect / limitations: No deletion or unrestricted sync. Current public-bucket policy excludes private financial files; binary plugin transfer remains unverified.

Code: object_storage.py:transfer; docs/cloudflare-r2-plugin.md

P28 Inspect a failed transfer and retry only storage

Read transfer <receipt_id>, explain completed/failed/unattempted objects and available checksum/byte observations. Retry the same approved transfer with a new ID, without repeating any accepted financial import. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Prior transfer receipt; for an archive retry use the already committed accepted result_run_id and manifest paths. Correct access/configuration first; do not mutate the old receipt.

Execution route: Workspace storage.status with transfer_receipt_id, then a new storage.pull/push request; CLI storage status --request-id <id>.

Native output formats: Old immutable receipt plus a new attempt receipt; independent module/transfer outcomes and progress. Existing workbook/input bytes remain unchanged for storage-only retries.

State effect / limitations: An existing ID returns its recorded outcome; rerunning FAILED with the same ID is not a new attempt.

Code: object_storage.py:execute/transfer_receipt; tests/test_transfer_failure_receipts.py

P29 Validate, export or replay a workbook

Validate committed run <latest_or_id>. Export a current workbook if requested, or replay run <run_id> into a separate XLSX output. Confirm record/numerical parity and distinguish replay from current analysis. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Committed state/manifests; explicit run ID for replay and operator output path. No new financial facts.

Execution route: CLI validate --run <latest_or_id>; export-excel --offline; replay --run-id <id> --output <path.xlsx> --offline.

Native output formats: Validation JSON with integrity/parity; export updates the main XLSX/run revision; replay writes the separate requested workbook.

State effect / limitations: Validation reads only; export creates derived publication; replay does not change accepted state. PDF presentation requires a separate reporting step.

Code: pipeline.py:validate/replay; excel.py:export_workbook/validate_workbook

P30 Activate an explicitly approved real dataset

After privacy and validation prerequisites pass, activate this supplied USER_IMPORT bundle as dataset <id>, recording author <name> and reason <reason>. Preserve the prior demo history and never relabel synthetic balances as real. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Full approved Pack A Bundle with real sourced records and dataset ID, privacy checks, author/reason and explicit real-data confirmation. Follow-on evidence/legacy/methodology gates remain separate.

Execution route: CLI activate-dataset --file <bundle.json> --author <name> --reason <reason> --confirm-real-data. No workspace request operation activates a dataset.

Native output formats: Real-dataset canonical snapshot, dataset transition/audit history, new workbook and run artifacts; source/history validation diagnostics.

State effect / limitations: GATED. Not currently ready; activation is not permission to publish private files into a public repository/bucket.

Code: cli.py:activate-dataset; pipeline.py:dataset boundary

P31 Review or import permitted workbook notes

Inspect the supplied Manual_Inputs sheet and import only confirmed permitted notes with author, reason, effective date and expiry. Reject numerical overrides. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: XLSX containing the exact Manual_Inputs header contract: override_id,ticker,field,value,author,reason,effective_date,expires_at,confirmed,supersedes. The only permitted field value is note. Author/reason, confirmation, effective date and optional expiry/supersession are separate metadata columns.

Execution route: CLI import-overrides --workbook <xlsx>.

Native output formats: Accepted notes/override lineage, workbook and run diagnostics; invalid fields are rejected.

State effect / limitations: Explicit note import; not a route to overwrite accepted prices, balances or models through Excel cells.

Code: excel.py:read_notes; schemas.py:NoteOverride; pipeline.py:import_overrides

P32 Run a budgeted refresh or measure caching

Refresh <workflow> as of <date> within <work units/seconds>, preserve a deferred-work queue and report actual usage. Separately benchmark unchanged and changed synthetic wealth inputs with parity verification. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Accepted input packs for the workflow; explicit budgets. Benchmark needs samples (1..10) and months (1..1200), not personal financial data.

Execution route: CLI run <workflow> --offline --as-of <date> --max-work-units <n> --max-seconds <n>; benchmark-wealth --samples <n> --months <n>.

Native output formats: Usage/deferred-work JSON and workbook diagnostics; benchmark JSON with timings, code/input hashes, parity, medians and ratios.

State effect / limitations: Budgeted run can publish PARTIAL/deferred results. Benchmark is isolated synthetic measurement, not guaranteed speedup or measured AI savings.

Code: runtime.py:RunContext; wealth_benchmark.py:benchmark

P33 Recover publication, submit pending requests and publish

Inspect pending publication recovery, recover the journal if needed, and process authorized committed requests. Publish to origin/work only after checking the expected remote head; report receipts and publication outcome. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Existing journal/request files, authorized publication policy and correct dataset/run IDs; clean checkout, repository access and private Actions context for the guarded scanner.

Execution route: CLI recover; request --pending --publish for activated private Actions; push --remote origin --branch work. Native push defaults to main, so specify work.

Native output formats: Recovery/run/request receipts, publication status/remote SHA and recovery bundle where configured. JSON/Markdown and workbook links as applicable.

State effect / limitations: GATED for production. No force-push or automatic workbook merge. GitHub-plugin publication uses inspected expected-head operations when native transport is unavailable.

Code: storage.py:publish/rollback_pending; workspace_requests.py:pending; cli.py:push

P34 Check automation prerequisites without starting schedules

Check whether <daily/valuation/ic/macro> can run under the current production policy as of <date>. List exact missing prerequisites and unconfirmed schedules. Do not enable jobs or invent alert methods. Deliver a downloadable PDF summary and link the native artifacts. Identify missing inputs without guessing.

Inputs needed / reuse: Committed production/schedule/source configuration, approved real data, legacy parity, provenance, required macro indicators, methods and manual-run evidence for eventual activation.

Execution route: CLI automation --workflow <workflow> --as-of <date> --dry-run; schedule-status; readiness.

Native output formats: Policy/readiness JSON with blocked requirements; schedule plan and alert-method status. PDF operational checklist.

State effect / limitations: Currently gated/disabled. Monday time and special monitors require confirmation; >8.0 scores need approved methodology. No generic scheduler is created by a status check.

Code: automation.py; readiness.py; config/schedules.json

6. Request submission templates

Required workspace envelope

Use requests/workspace/<request_id>.json. Required fields: request_id, operation, dataset_id, expected_run_id, as_of, authorized_by, instruction. Resolve dataset_id and published_run_id from committed canonical state. Optional workflows defaults to portfolio. Its eleven values are portfolio, valuation, update-dividends, timing, macro, watchlist, ic, retirement, cross-asset, daily and event.

Nine operations

run, simulate, accept-and-run, import, decision, validate, storage.pull, storage.push and storage.status. run cannot contain a changing input package. Accepted updates require a typed package or pinned package_artifact. decision accepts decisions/sources/cutoff only. Storage operations cannot update financial inputs.

Example: a screen request

The following is a template only. Replace dataset/run/user/date and choose a unique ID before submission. The instruction field has a 2000-character limit, so do not paste entire source documents there.

```
{
"schema_version": "1.0",
"request_id": "example-screen-001",
"operation": "run",
"dataset_id": "REPLACE_DATASET_ID",
"expected_run_id": "REPLACE_PUBLISHED_RUN_ID",
"as_of": "2026-11-01",
"authorized_by": "REPLACE_REQUESTING_USER",
"instruction": "Run the accepted valuation screen and report missing inputs",
"workflows": [
"valuation"
]
}
```

Input delivery choices

Inline: package is a typed WorkspacePackage with sources and optional portfolio/research/integration/wealth/financials/scenario_profiles/decisions/originals. Transferred JSON package: package_artifact uses receipt_id and artifact_id from a COMPLETE pinned pull. Do not combine inline and transferred packages. Originals use verified references with EvidenceDocument; downloading a PDF does not turn it into financial facts.

Storage payload example

```
{
"request_id": "example-pull-001",
"operation": "pull",
"objects": [
{
"key": "approved/REPLACE_EXACT_KEY.json",
"sha256": "REPLACE_64_HEX_SHA256"
}
]
}
```

Embed that storage payload in the workspace envelope with operation storage.pull, the same request ID and current dataset/run identity. A status request instead supplies transfer_receipt_id. Standalone storage CLI input is the StorageRequest object alone, without the outer workspace envelope.

Compatible older format

The legacy requests/wealth/<id>.json format remains available for validate/simulate/accept-and-run and retirement/cross-asset modules. Its package is WealthPacket (sources, wealth, optional cutoff). Do not confuse its modules property with the newer envelope's workflows property.

7. Outputs and PDF presentation rules

Core analytical publication

data/canonical/portfolio.json; workbooks/Investment_Workspace.xlsx; runs/<run_id>/manifest.json, input.json, snapshot.json, coverage.json, validation.json, changes.json and usage.json. Latest summary/readiness JSON and changes Markdown are under reports/latest. Additional valuation/integration summaries and timing/IC JSON feeds appear when relevant modules exist.

Workspace request outputs

reports/requests/<id>.json and .md contain status and links. Non-validation analytical requests also emit a scoped workbook and snapshot beneath reports/requests/<id>/. run/import/accept-and-run/decision can publish new canonical derived state; simulate does not accept candidate inputs. validate publishes diagnostics without a scoped workbook. Always inspect accepted_inputs_changed and individual module statuses.

Storage receipts

reports/storage/<id>.json records operation, immutable request hash, source head/run, objects/hashes/sizes and verification state. Failed transfers retain progress; storage.status reads their original outcome. Archives write a verified manifest last. Archive failure preserves valid financial calculation/import results. A transfer receipt is not a financial acceptance record.

Required PDF layer

For every requested report, the assistant must generate an actual PDF from verified outputs, supply a download link and a short chat summary. Include assessment/cutoff dates, dataset identity, module/transfer status, results, sources and assumptions, gaps and next action. Format money/rates/dates clearly and fit tables to pages. Keep XLSX/JSON/Markdown as supporting files. Never claim a PDF exists merely because the prompt requested one.

PDF capability limitation

The repository has no native PDF command/exporter or PDF output field in WorkspaceRequest. This catalogue's PDF is generated by the assistant's document tooling, outside the calculation pipeline. A separate reporting step is required for future report PDFs; adding a PDF path to archive_paths is invalid unless that file belongs to the applicable committed run manifest.

8. Exact required fields: schema reference

These field lists were extracted from the live Pydantic classes, including inherited required fields. They are not complete ready-to-import examples. Cross-record validators can impose additional conditional requirements, and optional/default fields can still be necessary for certification. Read the linked repository JSON schemas for types, enumerations, bounds and conditional behavior.

Source

source_id, title, publisher, effective_date, locator, evidence_label

Code: src/investment_workspace/schemas.py:18

Snapshot

timestamp, source_id, cash, positions

Code: src/investment_workspace/schemas.py:59

Transaction

transaction_id, timestamp, settlement_date, kind, source_id

Code: src/investment_workspace/schemas.py:74

Price

ticker, session_date, close, source_id

Code: src/investment_workspace/schemas.py:116

Earnings

record_id, ticker, effective_date, valid_until, source_ids, period_start, period_end, period_label, basis, reported_pat, weighted_average_shares, reported_eps, share_basis_date, quality_rationale

Code: src/investment_workspace/schemas.py:269

ValuationModel

record_id, ticker, effective_date, valid_until, source_ids, earnings_id, method, model_version, origin, methodology_rationale, forward_period, forward_basis, accounting_basis, share_basis_date, horizon_years, sovereign_benchmark, benchmark_date, equity_risk_premium, discount_rate_rationale, margin_of_safety, scenarios

Code: src/investment_workspace/schemas.py:333

Scenario

scenario_id, name, forward_eps, sustainable_dps, exit_price, horizon_dividends, exit_basis, dividends_rationale, discount_rate, terminal_growth, drivers

Code: src/investment_workspace/schemas.py:293

Dividend

record_id, ticker, effective_date, valid_until, source_ids, announcement_date, dps

Code: src/investment_workspace/schemas.py:378

FinancialFact

record_id, ticker, effective_date, valid_until, source_ids, period_start, period_end, accounting_basis, line_item, amount, units, locator

Code: src/investment_workspace/schemas.py:697

ScenarioProfile

record_id, ticker, effective_date, valid_until, source_ids, name, profile_version, financial_basis, period, horizon_years, assumptions

Code: src/investment_workspace/schemas.py:949

ScenarioLink

profile_id, financial_basis, period, horizon_years

Code: src/investment_workspace/schemas.py:964

BankForecast

period, accounting_basis, source_ids, fact_ids, fact_bindings, shares, tax_charge, engine, earning_assets, funding, fees, treasury_income, operating_costs, credit_charge, opening_equity, oci, capital_movements, opening_eligible_capital, eligible_oci, risk_weighted_assets, minimum_capital_ratio, distributable_reserves, cash_available, proposed_dividends

Code: src/investment_workspace/schemas.py:992

EPForecast

period, accounting_basis, source_ids, fact_ids, fact_bindings, shares, tax_charge, engine, commodities, royalties, fixed_operating_costs, depletion, net_interest, other_income, opening_receivables, collections, other_working_capital_increase, other_wc_excludes_receivables, capex, net_borrowing, cash_income_included

Code: src/investment_workspace/schemas.py:1026

IndustrialForecast

period, accounting_basis, source_ids, fact_ids, fact_bindings, shares, tax_charge, engine, products, fixed_operating_costs, depreciation, net_interest, other_income, receivables_increase, inventory_increase, payables_increase, capex, net_borrowing, midcycle_rationale, cash_income_included

Code: src/investment_workspace/schemas.py:1051

TimingPolicy

record_id, ticker, effective_date, valid_until, source_ids, benchmark, relative_lookback, momentum_lag, short_window, long_window, slope_sessions, flat_slope, liquidity_window, min_average_traded_value, min_rank_universe, max_extension, breakout_window, reversion_window, oversold_z, overbought_z, calibration_rationale

Code: src/investment_workspace/schemas.py:513

MarketHistory

record_id, ticker, effective_date, valid_until, source_ids, action_coverage_through, bars

Code: src/investment_workspace/schemas.py:471

RiskLimits

record_id, ticker, effective_date, valid_until, source_ids, max_position_weight, max_sector_weight, max_factor_weight, minimum_cash_weight, per_trade_risk_budget, max_daily_participation, correlation_lookback,

min_correlation_observations

Code: src/investment_workspace/schemas.py:578

WatchCandidate

record_id, ticker, effective_date, valid_until, source_ids, author, thesis, invalidation, portfolio_fit, desired_entry, stop_price, approach_fraction, next_review

Code: src/investment_workspace/schemas.py:620

ICProposal

record_id, ticker, effective_date, valid_until, source_ids, action, reason, confidence, invalidation, opportunity_cost

Code: src/investment_workspace/schemas.py:640

ICDecision

record_id, ticker, effective_date, valid_until, source_ids, proposal_id, decision, decided_by, decided_at, reason, reviewed_basis_hash

Code: src/investment_workspace/schemas.py:1070

WealthAsset

record_id, ticker, effective_date, valid_until, source_ids, economic_id, kind, available_from

Code: src/investment_workspace/schemas.py:789

WealthFlow

record_id, ticker, effective_date, valid_until, source_ids, kind, amount, price_basis, frequency, starts, ends

Code: src/investment_workspace/schemas.py:813

WealthScenario

record_id, ticker, effective_date, valid_until, source_ids, name, annual_inflation, returns

Code: src/investment_workspace/schemas.py:843

RetirementPlan

record_id, ticker, effective_date, valid_until, source_ids, start_date, retirement_date, months, monthly_spending_real, withdrawal_order

Code: src/investment_workspace/schemas.py:856

CrossAssetHistory

record_id, ticker, effective_date, valid_until, source_ids, asset_key, currency, return_basis, quotes

Code: src/investment_workspace/schemas.py:880

WorkbookMigration

record_id, ticker, effective_date, valid_until, source_ids, kind, workbook_sha256, mappings, approved_by, reason, store_permitted

Code: src/investment_workspace/schemas.py:766

WorkspaceRequest

request_id, operation, dataset_id, expected_run_id, as_of, authorized_by, instruction

Code: src/investment_workspace/workspace_requests.py:51

WorkspacePackage

(No unconditional required fields.)

Code: src/investment_workspace/workspace_requests.py:38

StorageRequest

request_id, operation

Code: src/investment_workspace/object_storage.py:22

EvidencePackage

(No unconditional required fields.)

Code: src/investment_workspace/intake.py:19

WealthRequest

request_id, operation, dataset_id, expected_run_id, as_of, instruction, authorized_by

Code: src/investment_workspace/wealth_requests.py:27

WealthPacket

wealth

Code: src/investment_workspace/wealth_requests.py:21

9. What to avoid promising

• No trade execution, automatic brokerage connection, money transfers or orders.

• No automatic acquisition of live PSX/news/macro data through the offline calculators. ChatGPT can use available connected tools separately; provenance and controlled import remain required.

• No automatic extraction/certification of financial figures from arbitrary PDFs or spreadsheets. No arbitrary formulas/scripts/paths in workspace requests.

• No fully implemented numerical forecasting for all registered sectors. Bank/E&P/Industrial are the typed first wave; remaining sectors retain their stated limitations.

• No Monte Carlo retirement success rate, generic portfolio optimizer, auto-minimum-contribution solver or guaranteed investment outcome.

• No configured production schedules or special alerts merely because code is present. Private publication, sources, legacy parity, methodology and manual production proof remain prerequisites.

• No direct plugin-backed Python/Actions R2 transport merely because the Cloudflare plugin is installed. Native runner configuration/credentials are separate, and binary plugin handling remains unverified.

• No private financial uploads to the public investmentcodex bucket. Respect the user's choice to leave public access enabled without silently changing that setting.

• No treating submitted files, passing tests, an R2 archive, a simulated result or an IC proposal as accepted real financial evidence.

Code and schema sources

Reviewed entry points, schemas and relevant implementation in src/investment_workspace/{cli,schemas,workspace_requests,wealth_requests,pipeline,portfolio,research,models,sector_engines,forecast_contracts,scenarios,dividends,decisions,workflows,timing,wealth,adapters,intake,migration,readiness,automation,object_storage,excel,wealth_benchmark}.py, relevant tests, JSON schemas, config files and phase-readiness documents. The source field on each prompt identifies the implementing entry point.

Operational snapshot is evidence dated 31 October 2026, not an assertion that external configuration can never change. Recheck current connected service settings and accepted dataset before acting.